

AUTHOR 3: Lameya Islam, Scrapped hotelworld.com, Participated in EDA.

```
!apt-get update
!pip install requests
!pip install beautifulsoup4
!pip install pandas
!apt install chromium-chromedriver
!cp /usr/lib/chromium-browser/chromedriver /usr/bin
!pip install chromedriver-autobrowser selenium-stealth
!pip install selenium
```

```

e)
_script("return document.body.scrollHeight")
t:

l data
, pages=7):

:
ge}"

.page_source, 'html.parser')
('div', class_='mfhotel-0')

.find('span', {'data-test': 'hotelName'})
('div', {'data-test': 'reviewBox'})
d('b').get_text(strip=True) if review_box and review_box.find('b') else 'N/A'
ox.find('p', class_='MuiTypography-root MuiTypography-body2 mfhotel-mgnx2q').get_text(strip=True) if review_box and review_bo:
h2', {'data-test': 'priceBox'})
xt(strip=True) if priceBox else 'N/A'
.find('span', {'aria-label': 'Refundable booking'})
efundable_span else 'No'
tel.select_one('span:nth-of-type(3)')
ddress_span.get_text(strip=True) if hotel_address_span else 'N/A'
'span', {'aria-label': lambda x: x and 'Star Hotel' in x and len(x) <= 12})
-label'] if star_span else 'N/A'
('src') for img in hotel.find_all('img', class_='MuiBox-root mfhotel-1se2dj0'))

{
otel_name_span.get_text(strip=True),
rating,
review_count,

efundable,

: hotel_address,
_images,
poair.com'

ls_data)

data)

data
es=10):

:
ge}"
ll_page(url, driver)
ource, 'html.parser')
('div', class_='hotel-info')

ds:
('div', class_='rt imgSizeVersionB')
nd('div', class_='multi-images')
src') for img in img_div.find_all('img')] if img_div else []
('span', class_='name')
ext().strip() if hotel_name else 'N/A'

find('a', class_='name')
et_text().strip() if hotel_name else 'N/A'

('div', class_='score')
v.find('span', class_='real') if rating_div else None
ting_span.get_text().strip() * 2, 1) if rating_span else 'N/A'
'div', class_='count')
iv.find('a') if count_div else None

```

```

nchor.get_text().replace(',', ' ') if review_anchor else 'N/A'
d('span', class_='highlight-tag')
efund_span and 'Free Cancellation' in refund_span.get_text() else 'No'
'div', id='meta-real-price')
find('span') if price_div else None
text().replace(",", " ").replace("US", "USD").replace("$", ' ') if price_span else 'N/A'
nd('p', class_='transport')
get_text().replace('Show on Map', '').replace(' ', ' ') if address_para else 'N/A'

ing,
{review_count}]",

dable,

dress,
s,
'

ls_data)

data)

ges=3):

s') # Run in headless mode (without GUI)
box')
-dev-shm-usage')

ns=options)

id detection

Engine",

:
current page
ge}"

y load

he page to trigger lazy loading
ow.scrollTo(0, document.body.scrollHeight);")
for lazy loaded content to appear

ifulSoup
.page_source, 'html.parser')

('div', class_='property-card')

l card and extract information

iv', class_='featured-container'):

div', class_='property-name')

iv.find('span').get_text(strip=True) if name_div.find('span') else name_div.get_text(strip=True)

nd('span', class_='distance-description')

```

```

get_text(strip=True) if address_span else 'N/A'

d('span', class_='score')

n.get_text(strip=True)
'NEW' else rt_txt

nd('span', class_='num-reviews left-margin')
get_text(strip=True) if reviews_span else 'N/A'

Selenium
'div', class_='carousel-list')
d('img') if image_div else None

o trigger loading

['src'] if img_tag else 'N/A'
with("data:image/gif;base64"):
_tag.get('data-v-lazy') or img_tag['src'] # Try to fetch lazy-loaded src

and private accommodation)
d_all('div', class_='property-accommodation-prices')
t price value

find('div', class_='tag-text')
Free Cancellation' in refundable_tag.get_text(strip=True):

1:
ices_divs[0].find('strong', class_='current')
ong_1.get_text(strip=True).replace('$', '') if price_strong_1 else None

# Take the first price if available
s) >= 2:
= prices_divs[1].find('strong', class_='current')
_strong_2.get_text(strip=True).replace('$', '') if price_strong_2 else 'N/A'
if price_2 else price # Fallback to the second price

({
_name,
ing,
iews,
",
dable,

dress,
l],
rld.com'

to a DataFrame
ata)

ndex=False)

```

```

heapoair.com/trip/hotels?ht=1&r1=a1ch0&ad=1&childs=0&rooms=1&fromdt=01%2F01%2F2025&todt=01%2F02%2F2025&dest=6145&modified=true
om/hotels/list?city=220&cityName=Dubai&provinceId=10965&countryId=9&districtId=0&checkin=2025%2F01%2F01&checkout=2025%2F01%2F01
.hostelworld.com/pwa/wds/s?q=Dubai,%20United%20Arab%20Emirates&country=Dubai&city=Dubai&type=city&id=1177&from=2025-01-01&to=
r_hotel(cheapoair_base_url, pages=PAGE_COUNT)
ip_base_url, pages=PAGE_COUNT)
(hostelworld_base_url, pages=PAGE_COUNT)

```

```

5
r_data_df, trip_data_df, hostelworld_df], ignore_index=True)

```

```

↩
0      Wyndham Dubai Deira      8.4
1      Pullman Sharjah      8.8
2      Mount Sina Hotel      N/A
3      Days Hotel By Wyndham Dubai Deira      7.9
4      The George Hotel by Saffron Dubai Creek      8.2
..      ...
953     Lotus Retreat Hotel      5.3
954     Hafez Hotel Apartments - Al Ras Metro Station      4.0
955     Signature inn Hotel Deira      4.9
956     Frank & Frank Vacation Homes      N/A
957     Al Manar Grand Hotel Apartment      N/A

      Review Count      Price Refundable      Stars \
0      (201 Reviews)      USD269      No      4 Star Hotel
1      (204 Reviews)      USD184      No      5 Star Hotel
2      N/A      USD92      No      1 Star Hotel
3      (194 Reviews)      USD166      No      3 Star Hotel
4      (204 Reviews)      USD160      No      3 Star Hotel
..      ...
953     (8)      $NT6,538      Yes      N/A
954     (2)      $NT3,309      Yes      N/A
955     (9)      $NT3,309      Yes      N/A
956     N/A      $NT2,409      Yes      N/A
957     N/A      $NT13,965      Yes      N/A

      Hotel Address \
0      Sherina Plaza Al Corniche Rd Dubai
1      Al Taawun Street, Sharjah
2      Al Sabkha Road, ae
3      One Deira Plaza Gold Souk Metro Station, Dubai
4      Baniyas Rd - Deira - Al Ras - Dubai, DUBAI
..      ...
953     - 8.9km from city centre
954     - 8.1km from city centre
955     - 8.9km from city centre
956     - 19km from city centre
957     - 6.1km from city centre

      Images      Source
0      [https://i.travelapi.com/lodging/58000000/5740...      Cheapoair.com
1      [https://i.travelapi.com/lodging/46000000/4596...      Cheapoair.com
2      [https://q-xx.bstatic.com/xdata/images/hotel/m...      Cheapoair.com
3      [https://i.travelapi.com/lodging/69000000/6887...      Cheapoair.com
4      [http://photos.hotelbeds.com/giata/bigger/05/0...      Cheapoair.com
..      ...
953     [https://a.hwstatic.com/image/upload/f_auto,q_...      HostelWorld.com
954     [https://a.hwstatic.com/image/upload/f_auto,q_...      HostelWorld.com
955     [https://a.hwstatic.com/image/upload/f_auto,q_...      HostelWorld.com
956     [data:image/gif;base64,R0lGODlhAQABAIAAAAAAP/...      HostelWorld.com
957     [https://a.hwstatic.com/image/upload/f_auto,q_...      HostelWorld.com

[958 rows x 9 columns]

```

Raw Hotel Data Count from Each Source

- This block shows a bar chart of raw hotel counts from each source.
- Data is grouped by the **Source** column.
- Visualizes how much data was scraped from **Cheapoair**, **Trip.com**, and **HostelWorld** before any processing.

```

import matplotlib.pyplot as plt

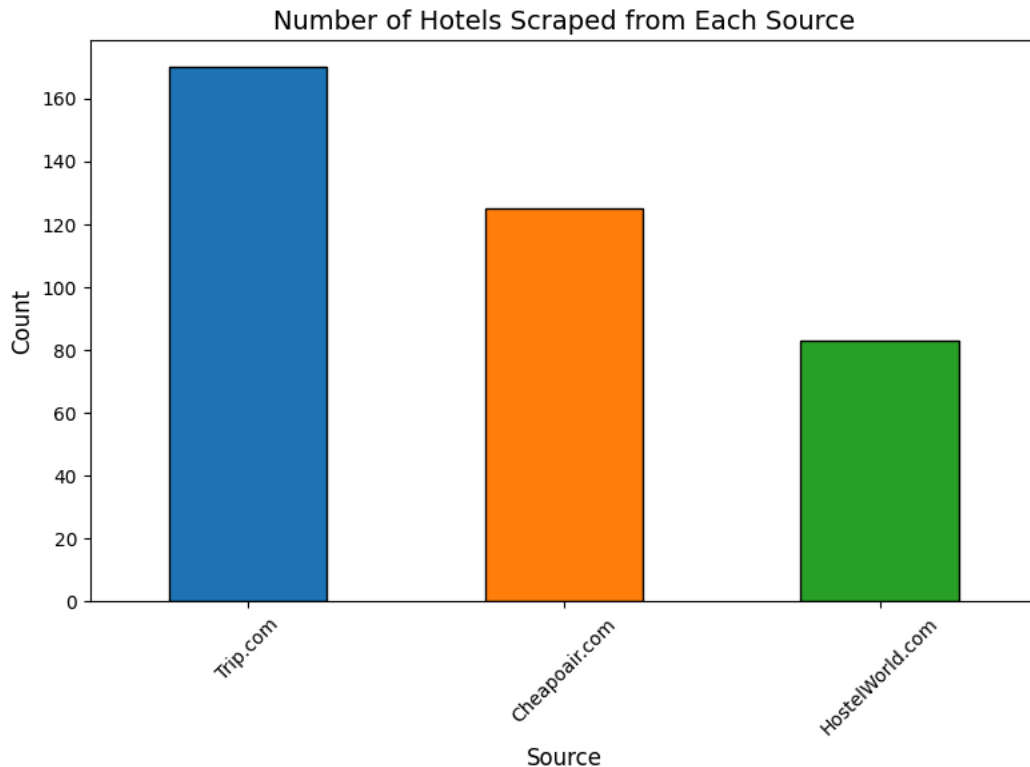
# Count the number of entries from each source
site_counts = hotel_data_df['Source'].value_counts()
print(site_counts)

# Plot a bar chart
plt.figure(figsize=(8, 6))
site_counts.plot(kind='bar', color=['#1f77b4', '#ff7f0e', '#2ca02c'], edgecolor='black')
plt.title('Number of Hotels Scraped from Each Source', fontsize=14)
plt.xlabel('Source', fontsize=12)
plt.ylabel('Count', fontsize=12)
plt.xticks(rotation=45)
plt.tight_layout()

plt.show()

```

```
Source
Trip.com      170
Cheapoair.com 125
HostelWorld.com 83
Name: count, dtype: int64
```



Step 2: Data Preprocessing

- Combined the dataframes from three websites into a single.
- Dropped the duplicate rows based on image
- Mapped Refundable column to binary value i.e, (Yes = 1, No = 0)
- Cleaned Price of symbols i.e, (\$, USD etc.) and converted to float
- Hotel Rating and Review Count converted to numeric format

```
hotel_data_df['Images'] = hotel_data_df['Images'].apply(tuple)
duplicate_count = hotel_data_df.duplicated().sum()

print(f'There are {duplicate_count} duplicate rows in the DataFrame.')
```

```
There are 130 duplicate rows in the DataFrame.
```

```
# Remove duplicates from the DataFrame
```

```
hotel_data_df_cleaned = hotel_data_df.drop_duplicates()
```

```
# Optionally, reset the index after dropping duplicates
hotel_data_df_cleaned.reset_index(drop=True, inplace=True)
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
df = pd.DataFrame(hotel_data_df_cleaned)
df['Hotel Rating'] = pd.to_numeric(df['Hotel Rating'], errors='coerce')
df['Review Count'] = df['Review Count'].astype(str).str.extract('(\d+)').astype(float) # Extract numerical part

#convert the price to float
df['Price'] = df['Price'].replace({r'[\$,USD,NT,/A]': '', r'\.': '', r'\s': '', r'\?': '', '\': ''}, regex=True)
df['Price'].replace('', np.nan,inplace=True)
```

```
df['Price'] = df['Price'].astype(float)
df.dropna(subset=['Price'], inplace=True)

# invalid_price_rows = df[pd.to_numeric(df['Price'].str.replace(r'[\$,USD,NT]', '', regex=True), errors='coerce').isna()]
# print(invalid_price_rows)
df['Refundable'] = df['Refundable'].map({'Yes': 1, 'No': 0}) # Convert to binary
df['Stars'] = df['Stars'].replace({'5 Star Hotel': 5, '4 Star Hotel': 4, '3 Star Hotel': 3, '2 Star Hotel': 2, '1 Star Hotel': 1})
```

 Show hidden output

STEP 3: Exploratory Data Analysis (EDA)

Used Summary Statistics `df.describe()` to quickly summarize key metrics like averages and ranges, helping identify trends and outliers.

```
# Summary Statistics
print(df.describe())
```



	Hotel Rating	Review Count	Price	Refundable	Stars
count	74.000000	74.000000	86.000000	86.000000	39.000000
mean	8.252703	234.527027	572.313953	0.302326	3.769231
std	0.865349	203.297961	2487.748513	0.461959	1.111224
min	4.600000	40.000000	10.000000	0.000000	1.000000
25%	7.800000	151.750000	153.000000	0.000000	3.000000
50%	8.400000	194.500000	207.500000	0.000000	4.000000
75%	8.800000	220.500000	312.250000	1.000000	5.000000
max	9.500000	1132.000000	22488.000000	1.000000	5.000000

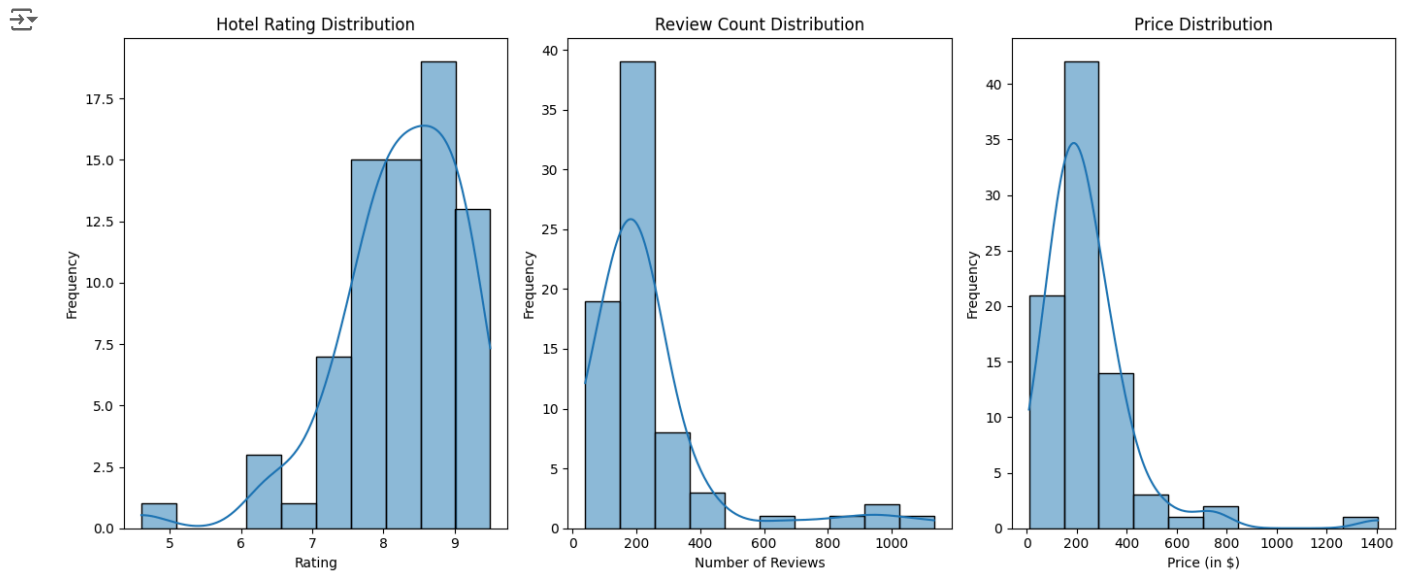
Used histograms to visualize the distribution of hotel ratings, review counts, and prices. It helps quickly understand the spread and frequency of these key metrics across the dataset.

```
# Histograms
plt.figure(figsize=(14, 6))
plt.subplot(1, 3, 1)
sns.histplot(df['Hotel Rating'], bins=10, kde=True)
plt.title('Hotel Rating Distribution')
plt.xlabel('Rating')
plt.ylabel('Frequency')

plt.subplot(1, 3, 2)
sns.histplot(df['Review Count'], bins=10, kde=True)
plt.title('Review Count Distribution')
plt.xlabel('Number of Reviews')
plt.ylabel('Frequency')

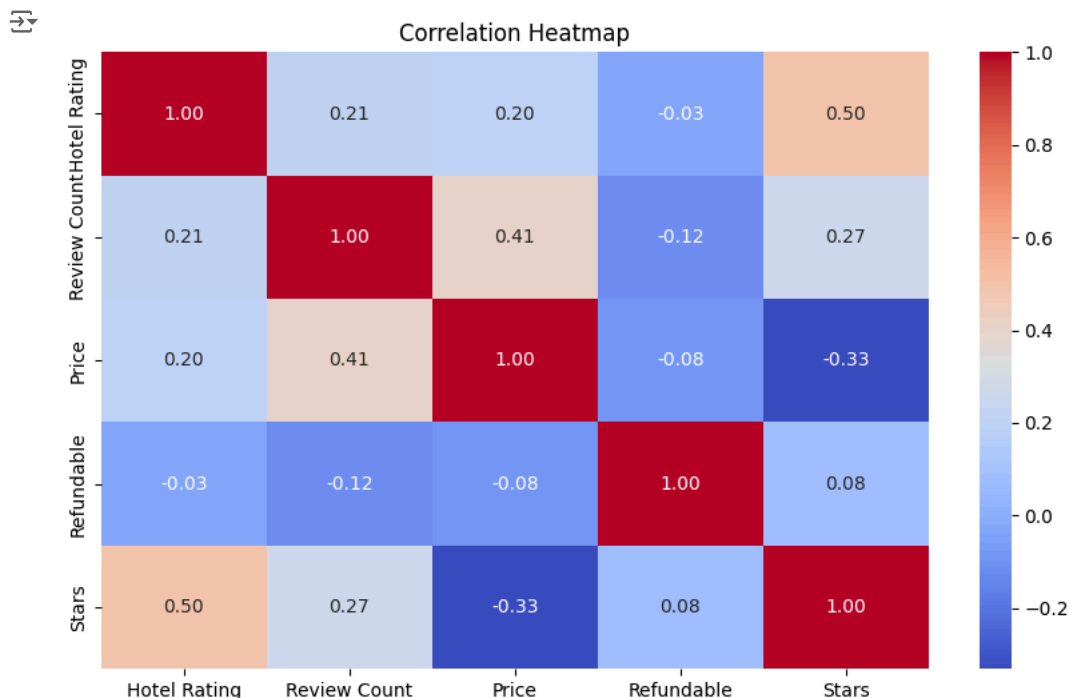
plt.subplot(1, 3, 3)
#removing extra skewed data from price comparisons to have meaningful EDA
price_df = df[df['Price'] <= 5000]
sns.histplot(price_df['Price'], bins=10, kde=True)
plt.title('Price Distribution')
plt.xlabel('Price (in $)')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```



Used correlation heatmap to visualize the relationships between numerical variables (e.g., rating, review count, price). It helps see which variables are strongly correlated, aiding in understanding potential connections within the dataset

```
# Correlation Heatmap
plt.figure(figsize=(10, 6))
correlation_matrix = df[['Hotel Rating', 'Review Count', 'Price', 'Refundable', 'Stars']].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.show()
```



Used Boxplots to compare hotel ratings based on start levels, with refundability as a distinguishing factor. It helps analyze how ratings vary by star classification and whether refundability impacts the ratings

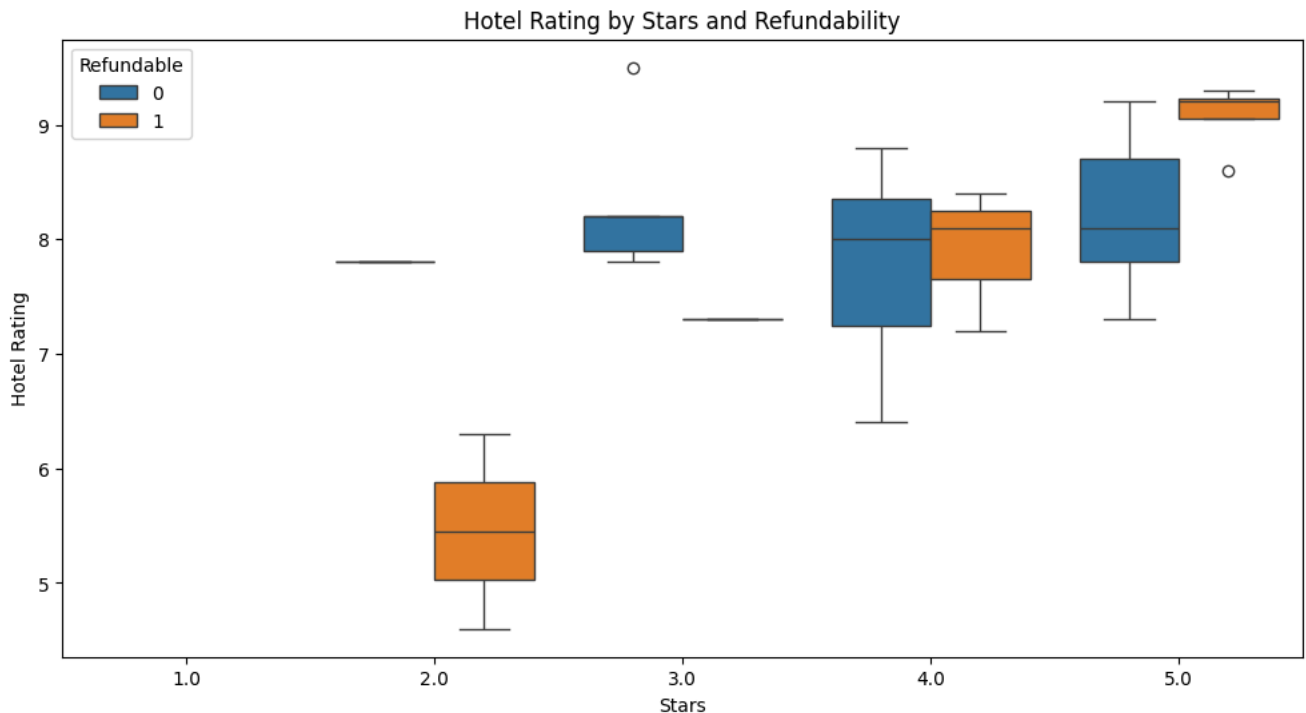
```
# Boxplots to compare ratings
plt.figure(figsize=(12, 6))
sns.boxplot(data=df, x='Stars', y='Hotel Rating', hue='Refundable')
plt.title('Hotel Rating by Stars and Refundability')
plt.xlabel('Stars')
```

```
plt.ylabel('Hotel Rating')
plt.legend(title='Refundable')
plt.show()
```

```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning: When grouping with a length-1 list-like, you
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640: FutureWarning: SeriesGroupBy.grouper is deprecated a
positions = grouped.grouper.result_index.to_numpy(dtype=float)
/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning: When grouping with a length-1 list-like, yo
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640: FutureWarning: SeriesGroupBy.grouper is deprecated a
positions = grouped.grouper.result_index.to_numpy(dtype=float)

```



```
print(df[['Stars', 'Hotel Rating', 'Refundable']].info())
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 86 entries, 0 to 85
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    Stars        39 non-null     float64
1    Hotel Rating  74 non-null     float64
2    Refundable    86 non-null     int64
dtypes: float64(2), int64(1)
memory usage: 2.7 KB
None

```

This section checks skewness in ratings, reviews, and prices, and performs a T-test to compare hotel ratings between refundable and non-refundable options. Results are interpreted based on the p-value.

```

# Skewness
print("Skewness of the Hotel Rating:", df['Hotel Rating'].skew())
print("Skewness of the Review Count:", df['Review Count'].skew())
print("Skewness of the Price:", df['Price'].skew())

# Inference Statistics: T-test for hotel ratings based on refundable status
from scipy import stats

refundable_ratings = df[df['Refundable'] == 1]['Hotel Rating'].dropna()
non_refundable_ratings = df[df['Refundable'] == 0]['Hotel Rating'].dropna()

t_stat, p_val = stats.ttest_ind(refundable_ratings, non_refundable_ratings)
print(f"T-test: t-statistic = {t_stat:.2f}, p-value = {p_val:.4f}")

# Interpretation of the T-test
if p_val < 0.05:
    print("There is a significant difference in hotel ratings based on refundable status.")
else:
    print("There is no significant difference in hotel ratings based on refundable status.")

```

```

Skewness of the Hotel Rating: -1.3415378791525285
Skewness of the Review Count: 2.82307470744564
Skewness of the Price: 8.384209510189633
T-test: t-statistic = -0.28, p-value = 0.7801
There is no significant difference in hotel ratings based on refundable status.

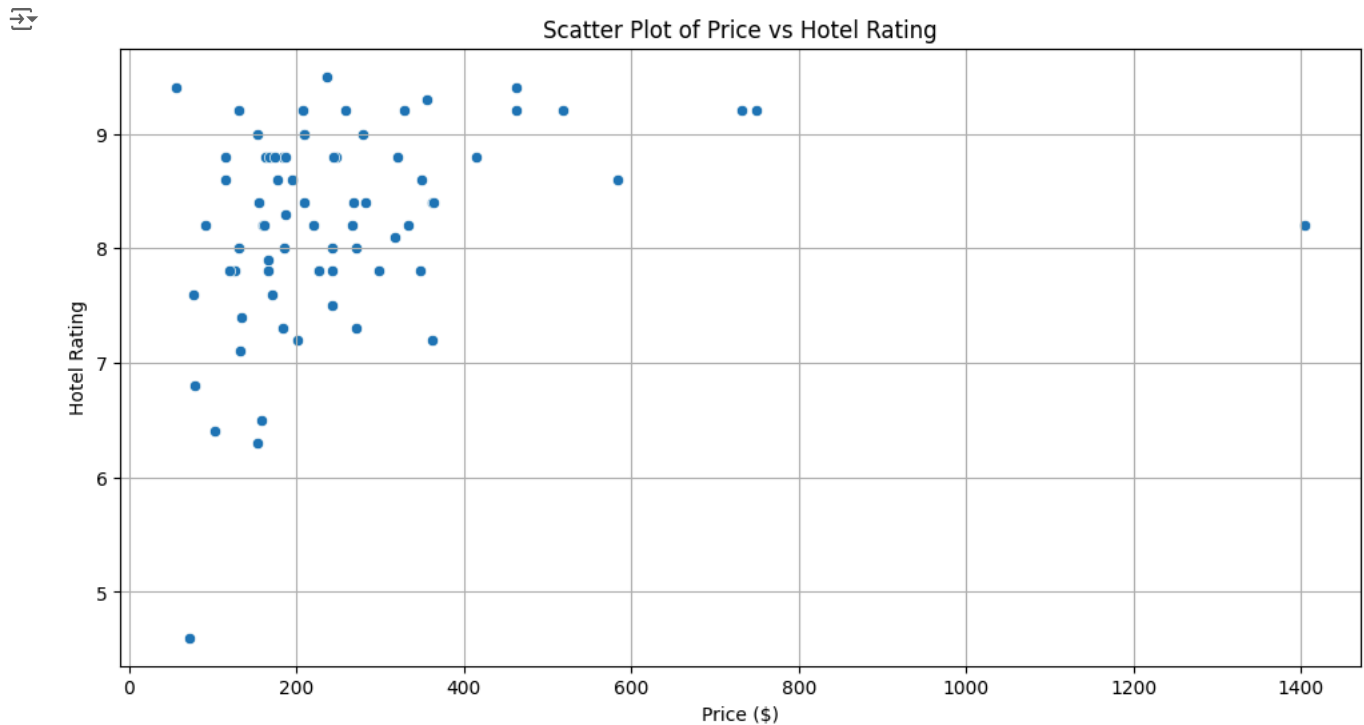
```

This scatter plot visualizes the relationship between hotel price and rating, helping to identify any trends or correlations between these two variables.

```

plt.figure(figsize=(12, 6))
sns.scatterplot(x='Price', y='Hotel Rating', data=price_df)
plt.title('Scatter Plot of Price vs Hotel Rating')
plt.xlabel('Price ($)')
plt.ylabel('Hotel Rating')
plt.grid()
plt.show()

```



This section categorizes hotel prices into defined bins and creates a boxplot to compare hotel ratings across different price ranges, helping to see how ratings vary with price levels.

```

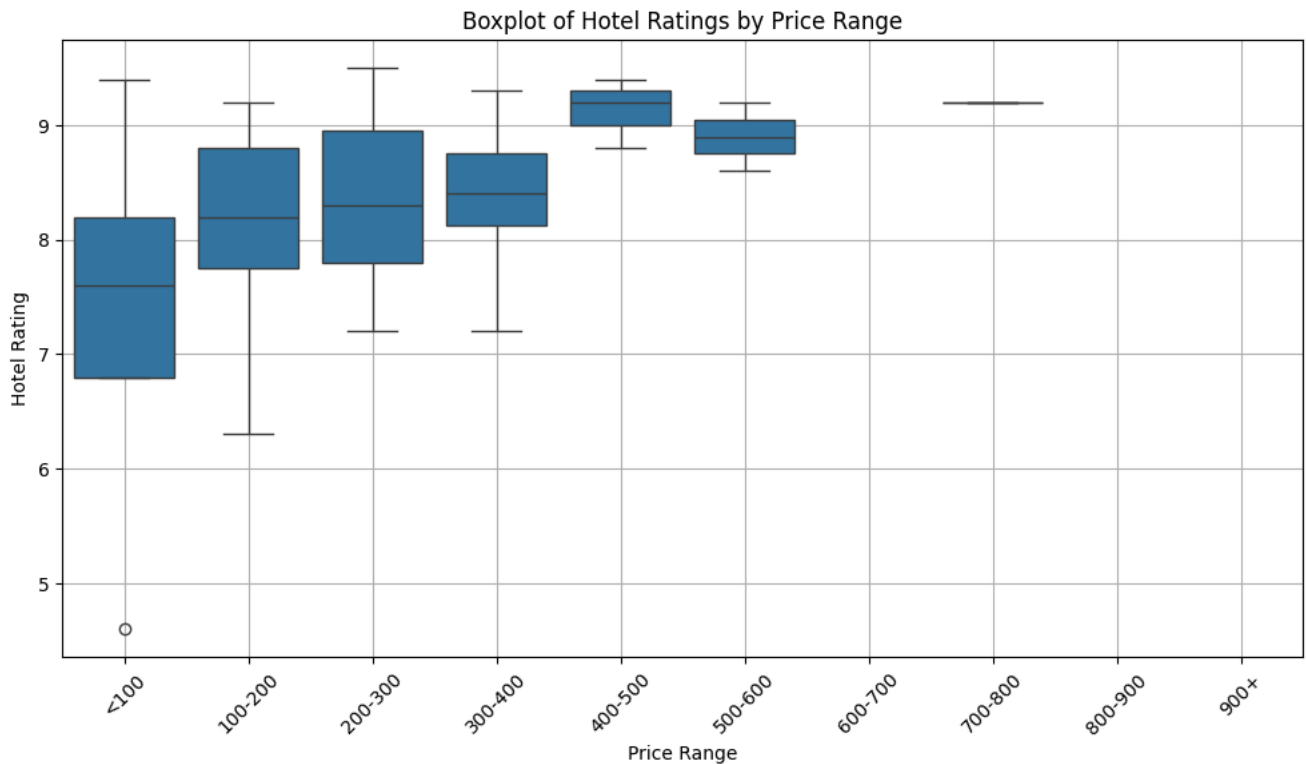
# Define price bins
bins = [0, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]
labels = ['<100', '100-200', '200-300', '300-400', '400-500', '500-600', '600-700', '700-800', '800-900', '900+']
price_df['Price Range'] = pd.cut(price_df['Price'], bins=bins, labels=labels)

plt.figure(figsize=(12, 6))
sns.boxplot(x='Price Range', y='Hotel Rating', data=price_df)
plt.title('Boxplot of Hotel Ratings by Price Range')
plt.xlabel('Price Range')
plt.ylabel('Hotel Rating')
plt.grid()
plt.xticks(rotation=45)
plt.show()

```

```
<ipython-input-125-84ddd5b7b6d3>:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

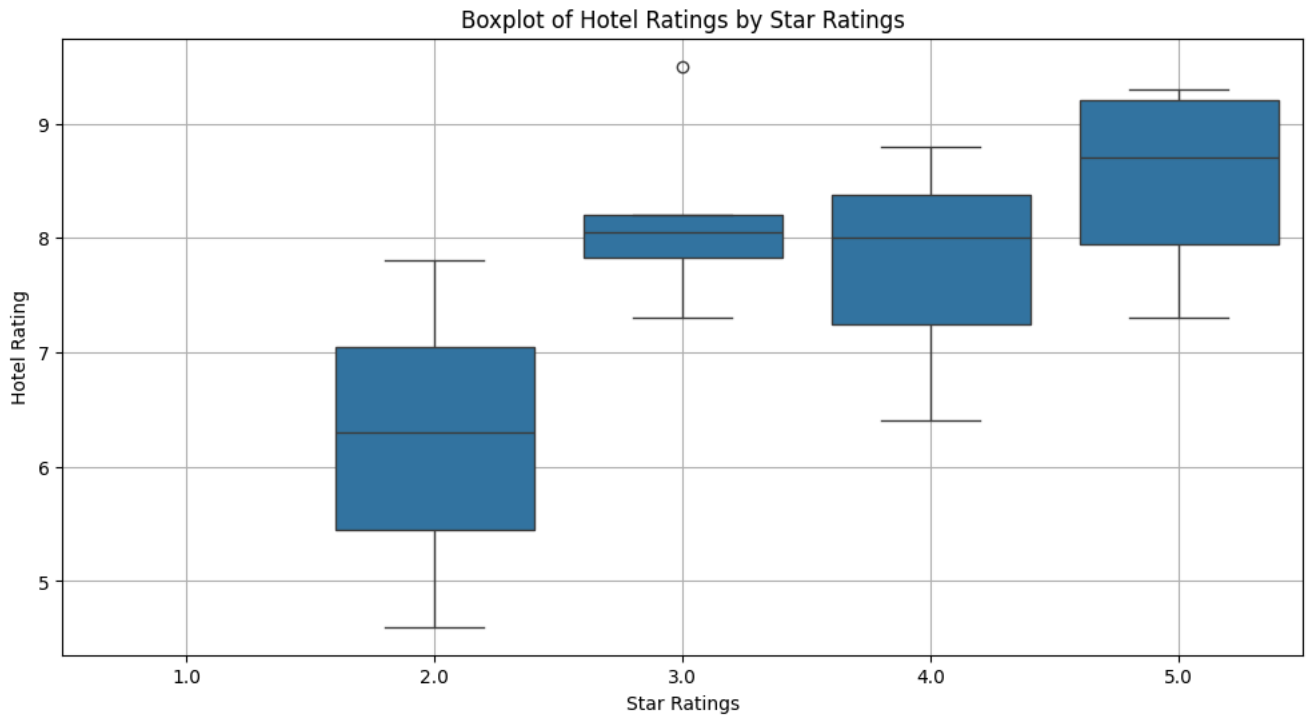
See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-price_df\['Price Range'\] = pd.cut\(price_df\['Price'\], bins=bins, labels=labels\)](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-price_df['Price Range'] = pd.cut(price_df['Price'], bins=bins, labels=labels))
 /usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640: FutureWarning: SeriesGroupBy.grouper is deprecated a
 positions = grouped.grouper.result_index.to_numpy(dtype=float)



This boxplot visualizes the relationship between hotel ratings and star ratings, helping us understand how ratings differ based on the number of stars a hotel has.

```
plt.figure(figsize=(12, 6))
sns.boxplot(x='Stars', y='Hotel Rating', data=df)
plt.title('Boxplot of Hotel Ratings by Star Ratings')
plt.xlabel('Star Ratings')
plt.ylabel('Hotel Rating')
plt.grid()
plt.show()
```

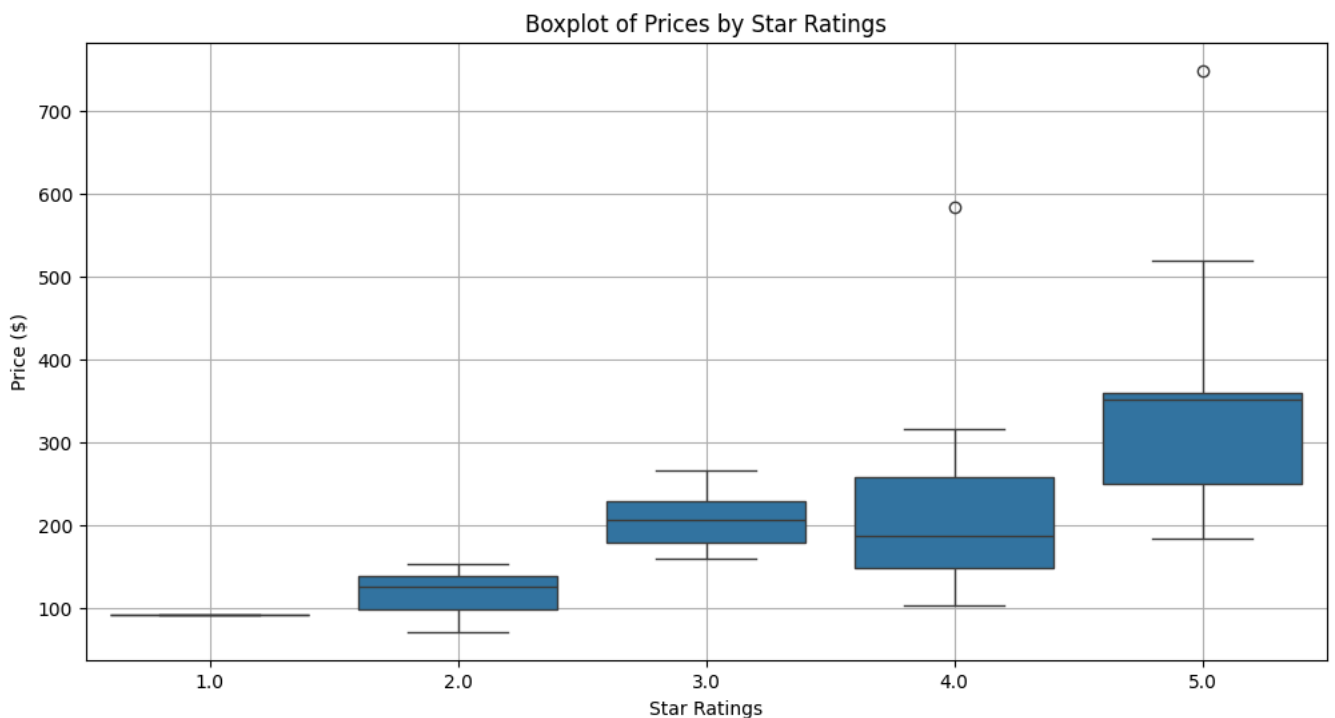
 /usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640: FutureWarning: SeriesGroupBy.grouper is deprecated a
positions = grouped.grouper.result_index.to_numpy(dtype=float)



This boxplot compares hotel prices across different star ratings, showing how the price of accommodations varies with the star rating.

```
plt.figure(figsize=(12, 6))
sns.boxplot(x='Stars', y='Price', data=price_df)
plt.title('Boxplot of Prices by Star Ratings')
plt.xlabel('Star Ratings')
plt.ylabel('Price ($)')
plt.grid()
plt.show()
```

 /usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640: FutureWarning: SeriesGroupBy.grouper is deprecated a
positions = grouped.grouper.result_index.to_numpy(dtype=float)



This section creates price ranges and filters hotels with 3-star and 4-star ratings. The two filtered DataFrames are then merged based on the price range to compare hotel ratings between 3-star and 4-star hotels within the same price range.

```
# Define price bins and labels
price_bins = [0, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]
price_labels = ['<100', '100-200', '200-300', '300-400', '400-500', '500-600', '600-700', '700-800', '800-900', '900+']

# Create a new column for price ranges
df['Price Range'] = pd.cut(df['Price'], bins=price_bins, labels=price_labels)

# Get the hotels with 3-star and 4-star ratings
three_star_hotels = df[df['Stars'] == 3].copy()
four_star_hotels = df[df['Stars'] == 4].copy()

# Merge the two DataFrames on 'Price Range'
common_hotels = pd.merge(
    three_star_hotels[['Price Range', 'Hotel Rating']],
    four_star_hotels[['Price Range', 'Hotel Rating']],
    on='Price Range',
    suffixes=('_3_star', '_4_star')
)

rating_comparison_stats = common_hotels.describe()
print(rating_comparison_stats)
```

```
↵
```

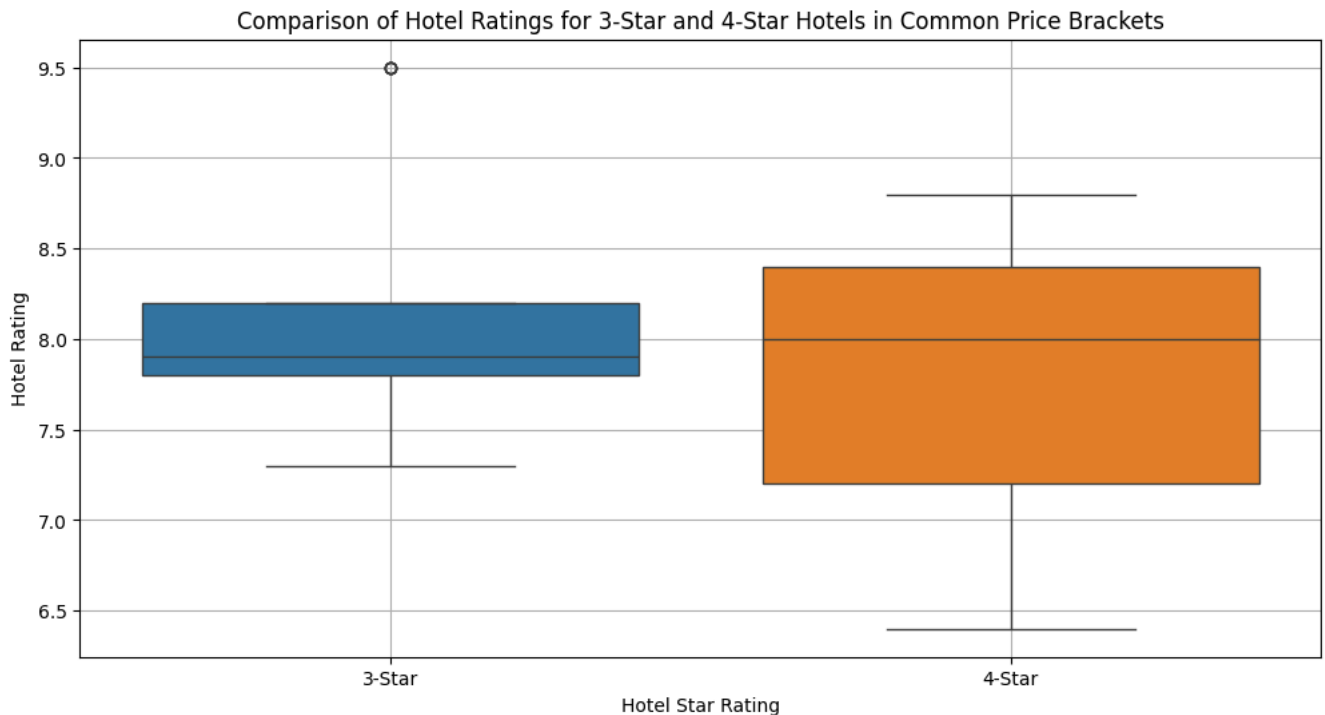
	Hotel Rating_3_star	Hotel Rating_4_star
count	39.000000	46.000000
mean	8.069231	7.780435
std	0.644472	0.737449
min	7.300000	6.400000
25%	7.800000	7.200000
50%	7.900000	8.000000
75%	8.200000	8.400000
max	9.500000	8.800000

```
plt.figure(figsize=(12, 6))
sns.boxplot(data=common_hotels[['Hotel Rating_3_star', 'Hotel Rating_4_star']])
plt.title('Comparison of Hotel Ratings for 3-Star and 4-Star Hotels in Common Price Brackets')
plt.xlabel('Hotel Star Rating')
plt.ylabel('Hotel Rating')
plt.xticks([0, 1], ['3-Star', '4-Star'])
plt.grid()
plt.show()
```

```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning: When grouping with a length-1 list-like, yo
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640: FutureWarning: SeriesGroupBy.grouper is deprecated a
positions = grouped.grouper.result_index.to_numpy(dtype=float)
/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning: When grouping with a length-1 list-like, yo
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640: FutureWarning: SeriesGroupBy.grouper is deprecated a
positions = grouped.grouper.result_index.to_numpy(dtype=float)

```



This section performs a T-test to compare hotel ratings between 3-star and 4-star hotels within the same price range. It then calculates the average ratings for both groups, grouped by price range, to further compare their mean ratings.

```

from scipy import stats

t_stat, p_value = stats.ttest_ind(
    common_hotels['Hotel Rating_3_star'].dropna(),
    common_hotels['Hotel Rating_4_star'].dropna()
)
print(f'T-test results: t-statistic={t_stat:.2f}, p-value={p_value:.3f}')

```

```

T-test results: t-statistic=1.91, p-value=0.060

```

```

# Calculate average ratings for 3-star and 4-star hotels within the same price range
average_ratings = common_hotels.groupby('Price Range').agg({
    'Hotel Rating_3_star': 'mean',
    'Hotel Rating_4_star': 'mean'
}).reset_index()

print(average_ratings)

```

```

Price Range  Hotel Rating_3_star  Hotel Rating_4_star
0          <100                NaN                NaN
1         100-200                7.8                7.328571
2         200-300                8.5                8.160000
3         300-400                NaN                NaN
4         400-500                NaN                NaN
5         500-600                NaN                NaN
6         600-700                NaN                NaN
7         700-800                NaN                NaN
8         800-900                NaN                NaN
9          900+                NaN                NaN

```

<ipython-input-131-9ff631b9d357>:2: FutureWarning: The default of observed=False is deprecated and will be changed to Tr

```

average_ratings = common_hotels.groupby('Price Range').agg({

```

This section calculates the skewness of 3-star and 4-star hotel ratings to assess the distribution shape, followed by a T-test to compare their average ratings for statistical significance.

```
# Calculate skewness
skewness_3_star = common_hotels['Hotel Rating_3_star'].dropna().skew()
skewness_4_star = common_hotels['Hotel Rating_4_star'].dropna().skew()
print(f'Skewness of 3-Star Ratings: {skewness_3_star:.2f}')
print(f'Skewness of 4-Star Ratings: {skewness_4_star:.2f}')
```

```
➦ Skewness of 3-Star Ratings: 1.13
Skewness of 4-Star Ratings: -0.47
```

```
t_stat, p_value = stats.ttest_ind(
    common_hotels['Hotel Rating_3_star'].dropna(),
    common_hotels['Hotel Rating_4_star'].dropna()
)
print(f'T-test results: t-statistic={t_stat:.2f}, p-value={p_value:.3f}')
```

```
➦ T-test results: t-statistic=1.91, p-value=0.060
```

Conclusion Average Ratings: If the average rating for 3-star hotels is higher than for 4-star hotels in a specific price range, it indicates greater satisfaction among guests for that category.

Distribution: If the ratings for 3-star hotels show less variability (lower standard deviation) and a more positive skew, it suggests consistent satisfaction.

Statistical Significance: If the p-value from the t-test is below a certain threshold (commonly 0.05), you can conclude that the difference in ratings between the two groups is statistically significant. But we need to gather more valid data to perform this analysis so its still in priliminary phase.

Step 4: User Interaction:

- **Input Preferences:** User specifies price range, rating, stars, and refund
- **Filter Hotels:** Hotels matching the user's preferences are filtered.
- **Sort Options:** User can sort by price, rating, or stars.
- **Display Results:** Filtered hotels are displayed with key details and images.
- **Hotel Details:** User can request detailed info about any hotel.
- **Exit:** User can exit anytime by typing 'exit'.

```
from IPython.display import display, HTML

def get_user_preferences():
    print("Please enter your preferences (or type 'exit' to quit):")
    min_price = input("Enter minimum price in $: ")
    if min_price.lower() == 'exit':
        return None

    max_price = input("Enter maximum price in $: ")
    if max_price.lower() == 'exit':
        return None

    min_review = input("Enter minimum Hotel Rating (1-10): ")
    if min_review.lower() == 'exit':
        return None

    min_stars = input("Enter minimum star rating (1-5) or type 'skip' to ignore: ")
    if min_stars.lower() == 'exit':
        return None

    refundable = input("Do you want refundable options only? (Yes/No): ").capitalize()
    if refundable.lower() == 'exit':
        return None

    return (
        float(min_price),
        float(max_price),
        float(min_review),
        min_stars.lower(),
        refundable
    )

def filter_hotels(min_price, max_price, min_review, min_stars, refundable):
```

```

    filtered_hotels = df[
        (df['Price'] >= min_price) &
        (df['Price'] <= max_price) &
        (df['Hotel Rating'] >= min_review)
    ]

    if refundable == 'Yes':
        filtered_hotels = filtered_hotels[filtered_hotels['Refundable'] == 1]

    if min_stars != 'skip':
        filtered_hotels = filtered_hotels[filtered_hotels['Stars'].notna() &
                                          (filtered_hotels['Stars'] >= int(min_stars))]

    return filtered_hotels

def get_sort_criteria():
    print("\nSort hotels by:")
    print("1. Price")
    print("2. Hotel Rating")
    print("3. Stars")
    sort_option = int(input("Enter the number corresponding to your choice: "))

    if sort_option == 1:
        sort_by = 'Price'
    elif sort_option == 2:
        sort_by = 'Hotel Rating'
    elif sort_option == 3:
        sort_by = 'Stars'
    else:
        print("Invalid choice. Defaulting to sorting by price.")
        sort_by = 'Price'

    return sort_by

def display_filtered_hotels(filtered_hotels):
    print("\nHere are the hotels based on your preferences:")

    # Add image HTML rendering
    filtered_hotels['Image'] = filtered_hotels['Images'].apply(
        lambda x: f'' if x else 'No Image Available'
    )

    # Select and reorder columns to display the desired output
    display_df = filtered_hotels[['Hotel Name', 'Price', 'Stars', 'Hotel Rating', 'Image']]

    # Render the DataFrame as HTML for displaying in Jupyter
    display(HTML(display_df.to_html(escape=False)))

def get_hotel_details(hotel_name):
    hotel = df[df['Hotel Name'] == hotel_name]

    if hotel.empty:
        print("Hotel not found!")
        return

    print("\nHotel Information:")
    print(f"Name: {hotel.iloc[0]['Hotel Name']}")
    print(f"Address: {hotel.iloc[0]['Hotel Address']}")
    print(f"Price: ${hotel.iloc[0]['Price']} per night")
    print(f"Hotel Rating: {hotel.iloc[0]['Hotel Rating']}/10")
    print(f"Stars: {hotel.iloc[0]['Stars']}")

    print("\nPhotos:")
    display_images(hotel.iloc[0]['Images'])

def display_images(photo_urls):
    image_html = ''.join(
        f''
        for url in photo_urls
    )
    display(HTML(image_html))

def display_image_placeholder(photo_urls):
    for url in photo_urls:
        print(f"[Image] Placeholder for image: {url}") # This simulates image display

def prompt_agent():
    preferences = get_user_preferences()
    if preferences is None:
        return # Exit if user types 'exit'

```

```
min_price, max_price, min_review, min_stars, refundable = preferences
filtered_hotels = filter_hotels(min_price, max_price, min_review, min_stars, refundable)

if filtered_hotels.empty:
    print("No hotels match your preferences.")
    return # Ask for preferences again

sort_by = get_sort_criteria()
sorted_hotels = filtered_hotels.sort_values(by=sort_by, ascending=True)

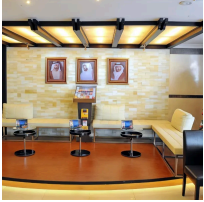
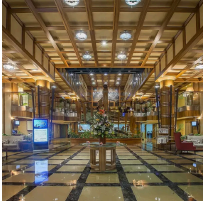
display_filtered_hotels(sorted_hotels)

prompt_agent()
```

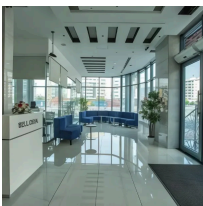
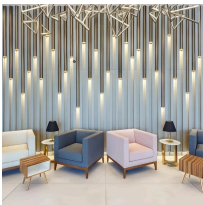
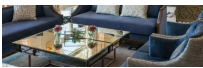
➦ Please enter your preferences (or type 'exit' to quit):
 Enter minimum price in \$: 1
 Enter maximum price in \$: 1000
 Enter minimum Hotel Rating (1-10): 1
 Enter minimum star rating (1-5) or type 'skip' to ignore: skip
 Do you want refundable options only? (Yes/No): Yes

Sort hotels by:
 1. Price
 2. Hotel Rating
 3. Stars
 Enter the number corresponding to your choice: 2

Here are the hotels based on your preferences:

	Hotel Name	Price	Stars	Hotel Rating	Image
68	Signature Inn Hotel - Free Parking	87.0	NaN	5.4	
57	Ramee Royal Hotel, Dubai	73.0	NaN	6.0	
75	Capitol Hotel Jumeirah Dubai	131.0	NaN	6.6	
97	Route 66	27.0	NaN	7.3	
58	Vintage Grand Hotel	77.0	NaN	7.6	
101	Brand New Boys Hostel	10.0	NaN	7.6	
52	Holiday International Hotel – Embassy District	167.0	NaN	7.8	
55	Al Habtoor Polo Resort	348.0	NaN	7.8	

67	The George Hotel by Saffron, Dubai Creek	126.0	NaN	7.8
94	Next'House - Business Bay Central Hostel	63.0	NaN	8.1
104	Dubai Town JBR	46.0	NaN	8.3
78	S19 Hotel-AI Jaddaf Metro Station	210.0	NaN	8.4
72	City Avenue Al Reqqa Hotel	177.0	NaN	8.6
69	Aloft Dubai Airport	191.0	NaN	9.0
89	Next'House - JBR Beach Hostel	38.0	NaN	9.1
87	Papaya Backpackers	78.0	NaN	9.3
90	Comfort Bed Inn	49.0	NaN	9.3



84 Lakeside Hostel Downtown 22.0 NaN 9.4



85 Alphatel Beach Hostel JBR 37.0 NaN 9.6



95 Lunar Hostel 30.0 NaN 9.6



93 Urban Panda Skyview Hostel JBR 182.0 NaN 9.7



83 Torch 77 55.0 NaN 9.7



98 Urban Panda Beach Hostel JBR 208.0 NaN 9.8

